

MACHINE LEARNING ENGINEERING

FASE 4 | AULA 01 -

ALGORITMOS CLÁSSICOS E PERCEPTRON

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	4
O QUE VOCÊ VIU NESTA AULA?	18
REFERÊNCIAS.....	19

EMSE

O QUE VEM POR AÍ?

Boas-vindas a mais um capítulo fascinante da nossa jornada pelo mundo das redes neurais e aprendizado de máquina. Neste material, vamos explorar a base das redes neurais, focando nas redes de camadas únicas e seus algoritmos clássicos. Você está prestes a descobrir como o Perceptron, um dos primeiros e mais fundamentais tipos de rede neural, funciona e como ele se relaciona com algoritmos clássicos que você já conhece.

Imagine um aluno ou aluna que está apenas começando a aprender sobre redes neurais. Ele(a) pode se sentir intimidado(a) pela complexidade aparente do tema. No entanto, ao entender o Perceptron de camada única, poderá enxergar a simplicidade e a elegância por trás dessa poderosa ferramenta de aprendizado. Nesta aula, vamos desmistificar esses conceitos, mostrando como eles são construídos a partir de algoritmos simples e clássicos.

Prepare-se para uma imersão teórica e prática que vai expandir seu entendimento sobre o campo das redes neurais e preparar você para enfrentar desafios mais complexos no futuro. Vamos começar!

HANDS ON

Boas-vindas à nossa seção prática, na qual colocaremos em ação tudo o que aprendemos sobre redes neurais de camadas únicas e o Perceptron. Nas videoaulas, você encontrará detalhes que explicam o passo a passo de como implementar e testar algoritmos clássicos, bem como o Perceptron de camada única. Por meio de exemplos práticos, as videoaulas ajudarão na consolidação da teoria.

SAIBA MAIS

Durante nossas aulas, revisaremos os algoritmos clássicos, como regressão linear e k-nearest neighbors, e compararemos suas performances com a do Perceptron. Você verá que ele funciona como uma rede neural fundamental e entenderá suas limitações em comparação com redes de camadas múltiplas. Prepare-se para uma imersão que solidificará seus conhecimentos e habilidades em redes neurais. Veja como é simples implementar o Perceptron com Python:

```
# Importar bibliotecas necessárias
import numpy as np
from sklearn.linear_model import Perceptron
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Gerar dados de exemplo
X = np.array([[2, 3], [1, 5], [2, 1], [3, 6], [5, 7], [4, 2]])
y = np.array([0, 0, 0, 1, 1, 1])

# Dividir dados em conjuntos de treinamento e teste
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
                                                    random_state=42)

# Treinar o Perceptron
perceptron = Perceptron(max_iter=1000, tol=1e-3)
perceptron.fit(X_train, y_train)

# Fazer previsões e avaliar o modelo
y_pred = perceptron.predict(X_test)
print(f'Accuracy: {accuracy_score(y_test, y_pred)}')
```

Código-fonte 1 – Implementando o Perceptron com Python

Fonte: elaborado pelo autor (2024)

O Perceptron, conforme descrito no texto, é um dos modelos mais importantes na história do cálculo neural e pode ser visto como uma rede neural de camada única. Ele funciona como um modelo simplificado de disparo neuronal, em que um neurônio

dispara (ativado) somente se a entrada ponderada total excede um limiar, neste caso zero. Matematicamente, a função de ativação do Perceptron é uma função degrau que retorna 1 se a entrada ponderada for maior que 0 e 0 caso contrário.

O Perceptron foi desenvolvido por Frank Rosenblatt em 1962, que também propôs um algoritmo de treinamento específico para ele. Uma das propriedades notáveis deste algoritmo é que, se existir um conjunto de valores de peso que possa classificar perfeitamente os dados de treinamento, o algoritmo do Perceptron é garantido para encontrar essa solução em um número finito de passos.

No entanto, apesar de sua simplicidade e eficácia inicial, o modelo do Perceptron tem limitações significativas, como foi formalmente provado por Minsky e Papert em 1969. Eles demonstraram que redes de Perceptron de camada única são incapazes de resolver problemas não linearmente separáveis, como a função XOR. Isso limita severamente a capacidade do modelo de lidar com complexidades presentes em dados reais.

Por causa dessas limitações, o interesse nos modelos de redes neurais (incluindo o Perceptron), diminuiu durante os anos 1970 e início dos anos 1980, até que novas abordagens e algoritmos para redes de múltiplas camadas foram desenvolvidos, revivendo o campo com o que hoje chamamos de “Perceptrons Multicamadas” ou “Redes MLP” (Multilayer Perceptron). É importante notar que é possível comparar os resultados de um perceptron com outros algoritmos clássicos, como forma de ilustrar suas limitações:

```
# Importar bibliotecas necessárias
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Perceptron, LogisticRegression
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score, classification_report

# Carregar o conjunto de dados Iris
data = load_iris()
X = data.data
y = data.target

# Dividir os dados em conjuntos de treinamento e teste
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
                                                    random_state=42)

# Padronizar os dados
scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Treinar e avaliar o Perceptron
perceptron = Perceptron(max_iter=1000, tol=1e-3)
perceptron.fit(X_train, y_train)
y_pred_perceptron = perceptron.predict(X_test)
print("Perceptron Accuracy:", accuracy_score(y_test, y_pred_perceptron))
print("Perceptron Classification Report:\n", classification_report(y_test,
y_pred_perceptron))

# Treinar e avaliar a Regressão Logística
log_reg = LogisticRegression(max_iter=1000)
log_reg.fit(X_train, y_train)
y_pred_log_reg = log_reg.predict(X_test)
print("Logistic Regression Accuracy:", accuracy_score(y_test,
y_pred_log_reg))
print("Logistic Regression Classification Report:\n",
classification_report(y_test, y_pred_log_reg))

# Treinar e avaliar o XGBoost
xgb = XGBClassifier(use_label_encoder=False, eval_metric='mlogloss')
xgb.fit(X_train, y_train)
y_pred_xgb = xgb.predict(X_test)
print("XGBoost Accuracy:", accuracy_score(y_test, y_pred_xgb))
print("XGBoost Classification Report:\n", classification_report(y_test,
y_pred_xgb))
```

Código-fonte 2 – Código para comparação de performance em modelos distintos

Fonte: elaborado pelo autor (2024)

Neste código, comparamos três modelos de classificação - Perceptron, Regressão Logística e XGBoost - utilizando o conjunto de dados Iris, que é comumente usado para tarefas de classificação. Inicialmente, os dados são divididos em conjuntos de treinamento e teste e padronizados para garantir que todas as características tenham a mesma escala.

Em seguida, cada modelo é treinado com o conjunto de treinamento e avaliado no conjunto de teste, com suas acurácias e relatórios de classificação sendo impressos para análise. Os resultados fornecem uma visão clara de como cada modelo performa em termos de precisão e outras métricas de classificação no conjunto de dados Iris.

O Perceptron de camada única é um modelo fundamental em redes neurais e serve como base para entender algoritmos mais complexos de deep learning. Sua estrutura é simples, consistindo em um conjunto de entradas que são multiplicadas por pesos, somadas e passadas por uma função de ativação para produzir a saída.

A função de ativação é crucial, pois adiciona não linearidade ao modelo, permitindo que o Perceptron aprenda padrões mais complexos do que meras funções

lineares. Exemplos comuns de funções de ativação incluem a sigmoide, que limita a saída entre 0 e 1, tornando-a útil para problemas de classificação binária, e a tangente hiperbólica, que produz saídas entre -1 e 1 e é útil em camadas ocultas de redes neurais por modelar fortemente saídas negativas e positivas de forma equilibrada.

Em termos de regressão linear, o Perceptron pode ser visto como uma aproximação, em que a função de ativação linear seria equivalente à hipótese linear em regressão. Já na classificação, ele é comparável ao classificador binário, em que a função de ativação, como a sigmoide, decide a classe de uma entrada com base em se a saída é maior ou menor que um limiar.

No entanto, o perceptron de camada única tem limitações significativas, principalmente a incapacidade de resolver problemas que não são linearmente separáveis, como o problema XOR, que não pode ser resolvido por uma única linha de decisão linear. Isso é parcialmente superado pelo perceptron de múltiplas camadas, que pode aprender representações mais profundas e complexas.

Conforme discutido nas definições 1.1 e 1.2 do livro "Mathematical Aspects of Deep Learning", a estrutura de aprendizagem de um Perceptron pode ser amplamente descrita usando o conceito de minimização do erro empírico, em que os pesos são ajustados para minimizar a discrepância entre as previsões e os verdadeiros rótulos dos dados de treinamento. Este método está alinhado com a abordagem clássica da teoria do aprendizado estatístico, que envolve ajustar um modelo aos dados de tal forma que o erro sobre os dados não vistos (erro de generalização) seja minimizado.

Aspectos Matemáticos do Perceptron de Camada Única

O perceptron de camada única pode ser descrito de forma eficaz usando as definições 1.1 e 1.2 da referência "Mathematical Aspects of Deep Learning", que estruturam a base teórica para a aprendizagem em redes neurais. Vamos explorar como essas definições se aplicam ao perceptron de camada única, destacando sua implementação e limitações.

Definição 1.1: Tarefa de Aprendizagem

Na tarefa de aprendizagem, um algoritmo é empregado para encontrar uma função dentro de um conjunto de hipóteses que minimize a perda sobre os dados de treinamento e que generalize bem para novos dados. No contexto do perceptron de camada única, definimos:

- Z como o espaço de dados, em que cada $z \in Z$ é um par de características de entrada e saída (x, y) .
- F como o conjunto de hipóteses, que no caso do perceptron de camada única consiste em todas as funções lineares parametrizadas pelos pesos θ .
- L como a função de perda, tipicamente a perda de erro quadrático para regressão ou a perda de entropia cruzada para classificação binária.

Matematicamente, a função de um perceptron pode ser expressa como:

$$f(x) = \sigma(w \cdot x + b)$$

em que σ é uma função de ativação não linear, w é o vetor de pesos, x é o vetor de entrada e b é o viés.

Definição 1.2: Tarefa de Predição

A tarefa de predição no Perceptron se concentra na classificação ou regressão baseada nos sinais de entrada. Ele recebe um vetor de entrada $x \in \mathbb{R}^d$ e produz uma saída que tenta prever o rótulo ou valor y associado. Para classificação binária, a função de ativação mais comum é a função sigmoide, que limita a saída entre 0 e 1, interpretada como probabilidade. A função do Perceptron para classificação pode ser expressa como:

$$f(x) = \sigma(w \cdot x + b) = \frac{1}{1 + e^{-(w \cdot x + b)}}$$

E para tarefas de regressão, uma função de ativação linear poderia ser usada (i.e., $\sigma(z) = z$), tornando o modelo equivalente a um regressor linear:

$$f(x) = w \cdot x + b$$

Caracterizando os principais conceitos do perceptron

O perceptron de camada única é um modelo de aprendizado de máquina que consiste basicamente em entradas, pesos, um viés e uma função de ativação. Vamos detalhar cada componente e explicar como eles se encaixam nos conceitos apresentados nas definições do livro "Mathematical Aspects of Deep Learning" (2023).

Entradas

As entradas (x) do perceptron representam as características ou atributos que são usados para fazer uma previsão ou classificação. Em termos matemáticos, x é um vetor em $\mathbb{R}^d$, em que d é o número de características. No contexto das definições, x corresponde ao espaço de características $X \in Z$.

Pesos e Viés

Os pesos w e o viés b são os parâmetros do modelo que são aprendidos durante o treinamento. Eles são ajustados para minimizar a função de perda L , que mede quão bem a saída do modelo $f(x)$ corresponde à saída desejada y . Matematicamente, os pesos são um vetor em $\mathbb{R}^d$ e o viés é um escalar (i.e. $b \in \mathbb{R}$).

Função de Ativação

A função de ativação σ é uma função não linear que é aplicada à soma ponderada das entradas e do viés. A escolha da função de ativação afeta diretamente a capacidade do modelo de capturar relações não lineares nos dados. No perceptron, funções comuns de ativação incluem a sigmoide (para classificação binária) e a linear (para regressão). A função de ativação transforma a combinação linear dos pesos e das entradas em uma saída que é usada para fazer previsões ou classificações.

Combinando os Componentes

Quando uma entrada x é fornecida ao perceptron, ela é multiplicada pelos pesos w , adicionada ao viés b e essa soma é então passada pela função de ativação σ . Conforme anteriormente apresentado, isso é expresso como:

$$f(x) = \sigma(w \cdot x + b)$$

Encaixe nos Conceitos de Aprendizado

1. **Espaço de Dados Z :** como definido, Z é composto por pares (x, y) , em que x são as características de entrada e y é a saída desejada (ou esperada, mais disso adiante). No treinamento, o modelo usa esses pares para aprender os parâmetros w e b .
2. **Conjunto de Hipóteses F :** cada configuração diferente de w e b define uma função diferente f dentro do espaço de hipóteses F . Pense em F como todas as relações possíveis entre x e y . O objetivo do treinamento é encontrar a

configuração de w e b que minimiza a perda entre as previsões do modelo e as saídas reais y .

3. **Função de Perda L :** a função de perda avalia o quão bem as previsões do modelo correspondem às saídas reais. A escolha da função de perda depende do tipo de tarefa (e.g., classificação ou regressão), mas sua escolha deve garantir que as condições analíticas de garantia de mínimo (diferenciabilidade e convexidade) são minimamente satisfeitas.

Portanto, o perceptron de camada única é um modelo fundamental que incorpora esses componentes básicos de aprendizado de máquina, servindo como um ponto de partida para compreender modelos mais complexos em redes neurais profundas. Você pode fazer um paralelo com todos os modelos clássicos e entenderá que a ideia do Perceptron é fundamentalmente similar aos problemas de estimativas paramétricas.

Do Perceptron às Redes Neurais de Camada Única

Nas referências do capítulo 4 e 5 de "Deep Learning: Foundations and Concepts" (2024), as redes neurais de camada única são discutidas como fundamentos básicos para o entendimento de estruturas mais complexas em redes neurais profundas. Vamos explorar como essas redes (que essencialmente se baseiam no conceito de Perceptron), são aplicadas para resolver problemas de classificação e regressão.

Estrutura Básica

Redes neurais de camada única consistem, tipicamente, em um conjunto de entradas (nodos de entrada), uma camada de nodos de processamento (neurônios) e uma camada de saída. Cada neurônio na camada única recebe entradas ponderadas por pesos, que são somadas juntamente com um viés. Essa soma é então transformada por uma função de ativação, resultando na saída do neurônio.

Função de Ativação

A função de ativação desempenha um papel crucial em definir a capacidade de uma rede neural de modelar relações não lineares nos dados. Para problemas de regressão, funções de ativação lineares são frequentemente utilizadas, permitindo

que a rede produza uma gama de valores contínua que pode diretamente corresponder a um valor alvo.

Em contraste, para problemas de classificação, funções de ativação não lineares como a sigmoide (para classificação binária) ou a softmax (para classificação multiclasse) são preferidas. Estas transformam as saídas lineares dos neurônios em probabilidades, que são úteis para determinar a classe de entrada.

Aprendizado e Otimização

O aprendizado em redes de camada única envolve o ajuste dos pesos e vieses com o objetivo de minimizar uma função de perda. Esta função mede a discrepância entre as saídas da rede e os valores alvo verdadeiros. O processo de otimização, frequentemente realizado por meio de métodos como o “gradiente descendente”, ajusta iterativamente os parâmetros para reduzir o erro de previsão.

Problemas de Regressão

Em problemas de regressão, o objetivo é prever um valor numérico contínuo. Redes neurais de camada única com uma função de ativação linear podem ser eficazes para problemas de regressão simples, em que as relações entre as variáveis são predominantemente lineares. A função de perda mais comum para regressão é o erro quadrático médio, que penaliza a rede com base no quadrado da diferença entre a previsão e o valor real.

Redes neurais de camada única são uma forma simplificada de rede neural frequentemente aplicada a problemas de regressão linear, em que a relação entre as variáveis de entrada e saída pode ser aproximada por uma função linear. Essas redes são particularmente úteis em situações nas quais a complexidade dos dados é moderada e a relação entre variáveis independentes e dependentes é bem entendida e predominantemente linear.

Estrutura e Funcionamento

Em problemas de regressão, uma rede neural de camada única consiste tipicamente em um conjunto de entradas que são ponderadas por coeficientes (pesos), somadas e então passadas por uma função de ativação linear. A saída da rede é, portanto, uma combinação linear das entradas, definida pela seguinte fórmula:

$$y = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

em que:

- y é a saída predita.
- w_i são os pesos atribuídos a cada entrada x_i .
- b é o viés (ou intercepto).
- n é o número de entradas.

Função de Ativação Linear

Para a regressão, a função de ativação mais comum é a linear, em que a função de ativação não altera a soma ponderada recebida. Isso significa que a função de ativação é essencialmente a função identidade $f(x) = x$. Essa escolha reflete a natureza dos problemas de regressão, em que se espera que a saída seja um valor contínuo correspondente à combinação linear das entradas.

Aprendizado e Otimização

O aprendizado em redes de camada única para regressão envolve ajustar os pesos w e o viés b para minimizar a diferença entre as saídas preditas pela rede e os valores reais observados. A função de perda mais comum utilizada é o erro quadrático médio (MSE), dado por:

$$MSE = \frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2$$

em que:

- m é o número de amostras.
- y_i são os valores reais.
- $\hat{y}_i$ são as predições da rede.

Trade-off de Viés e Variância

Uma consideração importante no treinamento de modelos de regressão é o trade-off entre viés e variância. O viés é um erro devido a suposições simplistas no algoritmo de aprendizado. Modelos de alta tendência são frequentemente muito simplificados, com pouca flexibilidade para aprender a verdadeira relação nos dados (sob ajuste).

Por outro lado, a variância refere-se ao quanto o modelo é sensível a pequenas flutuações nos dados de treinamento. Modelos de alta variância são altamente flexíveis, mas podem modelar o ruído dos dados como se fosse parte da relação desejada (sobre ajuste). A formulação matemática do trade-off de viés e variância para o erro esperado quadrático é dada por:

$$E[(y - \hat{y})^2] = \text{Viés}^2 + \text{Variância} + \text{Erro Irreduzível}$$

Este trade-off é crucial para o design de redes neurais, uma vez que escolher a complexidade correta do modelo (número de neurônios e camadas) pode minimizar o erro geral de previsão, equilibrando entre adaptar-se suficientemente aos dados e manter a capacidade de generalização para dados não vistos.

Redes neurais de camada única são ferramentas suficientemente boas para problemas simples, como os de regressão linear, oferecendo um mecanismo direto para modelar relações lineares e fornecendo uma base para entender como a complexidade dos modelos afeta seu desempenho geral por meio do trade-off de viés e variância.

Problemas de Classificação

Para problemas de classificação, o objetivo é categorizar entradas em classes discretas. A utilização de funções de ativação como a sigmoide ou softmax permite que a rede emita uma probabilidade de cada classe, facilitando a decisão de classificação. A função de perda típica usada em problemas de classificação é a entropia cruzada, que mede o desempenho da rede em termos de quão bem as probabilidades previstas se alinham com as etiquetas verdadeiras.

No capítulo 5 de "Deep Learning - Foundations and Concepts" (2024), o foco se volta para a aplicação de redes neurais de camada única em problemas de classificação. Diferentemente da regressão, em que a saída é contínua, a classificação lida com saídas categóricas. Vamos explorar como essas redes são estruturadas e otimizadas para problemas de classificação, incluindo a escolha da função de ativação e a função de perda adequadas.

Estrutura de Redes Neurais para Classificação

Para problemas de classificação, uma rede neural de camada única consiste em várias entradas que são combinadas linearmente com pesos e um viés, seguidos

por uma função de ativação que é tipicamente não linear. A função de ativação transforma a combinação linear em uma forma que pode ser interpretada como uma probabilidade ou uma decisão categórica. As funções de ativação mais comuns para classificação incluem:

- **Sigmoide:** usada em classificação binária, limita a saída entre 0 e 1, representando a probabilidade de a entrada pertencer a uma das duas classes.
- **Softmax:** utilizada em classificação multiclasse, transforma as saídas em uma distribuição de probabilidade entre várias classes.

A estrutura geral da rede pode ser descrita pela fórmula:

$$\hat{y} = \sigma(W \cdot X + b)$$

em que:

- y é a saída predita.
- σ é a função de ativação.
- W são os pesos.
- X são as entradas.
- b é o viés.

Aprendizado e Otimização

O processo de aprendizado em redes de camada única para classificação também envolve ajustar os pesos e o viés para minimizar uma função de perda, que mede a discrepância entre as saídas preditas e as saídas reais. Para classificação, a função de perda mais comum é a entropia cruzada, que para a classificação binária é definida como:

$$L = -\frac{1}{N} \sum_{i=1}^N y_i \log \hat{y}_i + (1 - y_i) (1 - \log \hat{y}_i)$$

E para a classificação multiclasse com a função de ativação Softmax:

$$L = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{ic} \log \hat{y}_{ic}$$

em que:

- N é o número de amostras.
- C é o número de classes.
- y_i são os valores reais.
- $\hat{y}_i$ são as previsões da rede.

Viés e Variância em Classificação

Assim como na regressão, o trade-off de viés e variância é uma consideração crucial em classificação. Modelos com alto viés podem não capturar as complexidades dos dados e falhar na generalização, resultando em alto erro em novos dados (sob ajuste). Modelos com alta variância podem capturar ruído como se fossem características significativas dos dados, levando a um modelo que não se generaliza bem (sobre ajuste).

Redes neurais de camada única para classificação fornecem uma ferramenta fundamental para resolver problemas em que as saídas são categóricas. A escolha da função de ativação e da função de perda adequadas é crucial para o sucesso desses modelos. Embora essas redes sejam limitadas por sua simplicidade, particularmente em não conseguir capturar relações complexas sem a ajuda de múltiplas camadas, elas oferecem uma introdução valiosa ao mundo das redes neurais e estabelecem uma base para arquiteturas mais profundas e complexas.

Generalizações e Limitações

Apesar de sua simplicidade e eficácia em certos tipos de problemas, redes neurais de camada única têm limitações significativas, principalmente devido à sua incapacidade de capturar complexidades em dados que apresentam relações não-lineares complexas. Este é um motivo pelo qual arquiteturas de redes mais profundas, com múltiplas camadas ocultas, são frequentemente necessárias para tarefas mais complexas.

Redes neurais de camada única têm sido um ponto de partida crucial para o entendimento e desenvolvimento de modelos mais sofisticados de redes neurais. No

entanto, apesar de sua utilidade inicial e simplicidade, elas apresentam limitações significativas que devem ser consideradas ao aplicá-las a problemas do mundo real. Vamos explorar estas limitações, bem como as possíveis generalizações que podem ser derivadas de seu uso.

Limitações das Redes Neurais de Camada Única

1. **Capacidade de Modelagem Limitada:** a maior limitação das redes de camada única é sua incapacidade de capturar complexidades e padrões não lineares complexos nos dados. Como consistem apenas de uma camada linear seguida de uma função de ativação não linear, elas são essencialmente classificadores lineares ou regressores, limitados a superfícies de decisão simples.
2. **Problemas Não Linearmente Separáveis:** em problemas de classificação, por exemplo, redes de camada única não podem resolver categorizações que não são linearmente separáveis, como é exemplificado pelo clássico problema do XOR, em que as categorias não podem ser divididas por uma única linha reta ou superfície plana.
3. **Sobreajuste e Generalização:** embora o sobreajuste (overfitting) possa ocorrer em qualquer modelo de aprendizado de máquina, redes de camada única têm um espaço restrito de hipóteses que podem torná-las incapazes de generalizar bem amostras de treinamento para dados não vistos, especialmente se a distribuição dos dados for complexa.

Generalizações a Partir de Redes de Camada Única

1. **Fundamentos para Redes Profundas:** as redes de camada única servem como um modelo fundamental para o entendimento das redes neurais mais complexas. A compreensão de como os pesos e vies são ajustados, como as funções de ativação influenciam a saída do modelo e como minimizar uma função de perda são conceitos que se aplicam a todas as redes neurais, independentemente de sua complexidade.
2. **Introdução às Arquiteturas de Aprendizado Profundo:** trabalhar com redes de camada única oferece uma excelente introdução aos princípios de

aprendizado profundo, incluindo o treinamento com backpropagation, otimização de hiperparâmetros e técnicas de prevenção de sobreajuste, que são cruciais para o sucesso com arquiteturas mais profundas.

3. **Expansão para Modelos Mais Complexos:** a partir das redes de camada única, pesquisadores(as) e praticantes podem explorar a adição de camadas, o que leva ao desenvolvimento de redes multicamadas ou perceptrons multicamadas (MLPs) e, mais recentemente, redes Kolmogorov-Arnold (KANs). Essas redes podem modelar decisões e funções muito mais complexas, graças à sua capacidade de criar representações hierárquicas dos dados.

O QUE VOCÊ VIU NESTA AULA?

Nessa aula, aprendemos os conceitos básicos de redes neurais de camada única e o Perceptron, um dos primeiros e mais simples tipos de rede neural. Também vimos como ele funciona e como ele se compara com algoritmos clássicos de aprendizado de máquina, como regressão linear e k-nearest neighbors. Também testamos esses algoritmos em código, usando exemplos práticos e divertidos.

Discutimos os aspectos matemáticos do Perceptron de camada única usando as definições de tarefa de aprendizagem e tarefa de predição do livro "Mathematical Aspects of Deep Learning" e conhecemos os componentes do Perceptron, como entradas, pesos, vies e função de ativação, além de como eles se relacionam com os conceitos de espaço de dados, conjunto de hipóteses e função de perda.

Exploramos as aplicações de redes neurais de camada única para problemas de regressão e classificação, explicando a escolha da função de ativação e da função de perda mais adequadas para cada tipo de problema e falamos sobre o trade-off de vies e variância, que é uma questão importante no treinamento de modelos de regressão e classificação.

Finalmente, avaliamos as limitações das redes neurais de camada única, principalmente sua dificuldade de capturar complexidades e padrões não lineares nos dados e como essas limitações motivaram o desenvolvimento de redes de múltiplas camadas, que formam a base das redes neurais profundas que estão por trás de muitas das tecnologias que usamos no dia a dia.

A conclusão é que as redes neurais de camada única e o Perceptron são modelos fundamentais que servem como base para que possamos entender e desenvolver modelos mais sofisticados de redes neurais e que a compreensão desses conceitos é essencial para ampliar nosso conhecimento e nossas habilidades em redes neurais e aprendizado de máquina!

REFERÊNCIAS

- BISHOP, C. M.; BISHOP, C. **Deep Learning**: Foundations and Concepts. [s.l.]: Springer, 2024.
- GROHS, P.; KUTYNIOK, G. **Mathematical Aspects of Deep Learning**. [s.l.]: Cambridge University Press, 2023.
- HUYEN, C. **Designing Machine Learning Systems**: An Iterative Process for Production-Ready Applications. [s.l.]: Springer Science+Business Media, LLC, 2007.
- MURPHY, K. P. **Machine Learning**: A Probabilistic Perspective. [s.l.]: MIT Press, 2024.
- PRINCE, S. J. D. **Understanding Deep Learning**. [s.l.]: MIT Press, 2024.
- STOLL, E. et al. **Machine Learning and Systems Engineering**. [s.l.]: Springer, 2023.

PALAVRAS-CHAVE

Deep Learning. Redes Neurais. Perceptron. Redes Neurais de Camada Única.

EMSE